R.B. GOTHI JAIN COLLEGE FOR WOMEN

PROJECT: AI BLOGNEST API

TEAM MEMBERS

1. HASHVA VARDHINI.K - [TEAM LEADER]
2. ISHWARYA.S
3. AARTHIKA.S
4. JOTHILAKSHMI.V
5. HEMAPRIYA.V

Backend REST API Project Documentation

AI BlogNest API

Backend REST API Project Documentation

1. Introduction

AI BlogNest API is a backend application developed to manage blog posts through a RESTful API. The application is built using Node.js and Express.js, with MongoDB used as the database and Mongoose used to establish communication between the application and MongoDB.

The API provides operations for creating, viewing, updating, filtering, and deleting blog posts. Each blog can contain information such as title, content, author, category, tags, status, summary, and meta description. The application also maintains creation and update timestamps for blog records.

The backend is configured to accept JSON data and supports cross-origin requests using CORS. The project was tested successfully using a dedicated API test script that verifies the complete CRUD workflow.

2. Project Objective

The main objective of the AI BlogNest API is to develop a simple and efficient backend system for managing blog content.

The objectives are:

- To create a backend API for blog management.
- To store blog information in MongoDB.
- To implement CRUD operations.
- To provide API routes for accessing blog data.
- To update and delete existing blog records.
- To filter blog records based on available fields.
- To test the API operations using a test script.
- To maintain blog creation and modification timestamps.

3. Problem Statement

Managing blog information manually can make it difficult to create, update, retrieve, and delete content efficiently. A backend API provides a structured way to manage blog data and allows different applications or frontends to communicate with the database.

The AI BlogNest API addresses this requirement by providing REST API operations for managing blog posts through an Express.js backend connected to MongoDB.

4. Proposed System

The proposed system is a backend REST API that manages blog posts using Express.js and MongoDB.

The application receives requests from a client, processes them through Express routes, communicates with MongoDB using Mongoose, and returns the appropriate response in JSON format.

The major operations implemented in the system are:

- Create a blog
- Retrieve all blogs
- Retrieve a single blog
- Update a blog
- Filter blogs
- Delete a blog

5. Technologies Used

Node.js

Node.js is used as the runtime environment for executing the backend JavaScript application.

Express.js

Express.js is used to create the server and define the API routes.

MongoDB

MongoDB is used as the database for storing blog information.

Mongoose

Mongoose is used to connect the Node.js application with MongoDB and manage blog records.

CORS

CORS is enabled to allow requests from different origins.

dotenv

The dotenv package is used to load configuration values such as the MongoDB connection string from environment variables.

6. System Configuration

The application uses Express middleware for handling requests.

The following configurations are implemented:

- CORS middleware
- JSON request-body parsing
- Blog route configuration
- MongoDB connection
- Server port configuration

The MongoDB connection uses the MONGO_URI environment variable. If a port is not specified in the environment configuration, the application uses port 5000.

7. API Base Route

The blog API is configured under the following base route:

```
/api/blogs
```

The application also provides a root route:

```
/
```

When the root route is accessed, the server returns:

```
AI BlogNest API is running!
```

8. Blog Data

The blog records tested in the application contain fields such as:

- _id
- title
- content
- author
- category
- tags
- status
- summary
- metaDescription
- assets
- createdAt
- updatedAt

The database generates a unique _id for each blog record. The test results also show createdAt and updatedAt timestamps.

9. CRUD Operations

9.1 Create Blog

The Create operation is used to add a new blog to the database.

The test data included:

- Title: AI BlogNest Test Blog
- Content: This is a sample blog created using the API.
- Author: Hashvavardhini
- Category: technology
- Tags: AI, Node.js, MongoDB
- Status: published

The API successfully created the blog and returned its unique ID and timestamps.

9.2 Get All Blogs

The Get All operation retrieves the available blog records from the database.

The test output successfully returned multiple blog records, including their titles, content, authors, tags, status, and timestamps.

9.3 Get Single Blog

The Get Single operation retrieves one specific blog using its unique ID.

The test successfully returned the selected blog with its complete stored information.

9.4 Update Blog

The Update operation modifies an existing blog.

During testing, the blog was updated with:

- Updated title: AI BlogNest Updated Blog
- Updated content
- Updated tags: AI, Express, Updated
- Status: published

The updatedAt value was also changed after the update operation.

9.5 Filter Blogs

The Filter operation is used to retrieve blogs matching the specified filtering condition.

The test output successfully returned blog records from the database based on the filter used in the test.

9.6 Delete Blog

The Delete operation removes a selected blog from the database.

The test returned the following successful response:

```
Blog deleted successfully
```

After deletion, the final blog list was retrieved to verify the result.

10. API Testing

A separate test script was used to verify the API functionality.

The following sequence was tested:

1. Create Blog
2. Get All Blogs
3. Get Single Blog
4. Update Blog
5. Filter Blogs

6 Delete Blog

7 Get Final Blog List

The test output shows that all these operations were completed successfully.

11. Testing Result

The CRUD API tests were completed successfully.

The Create operation generated a new blog record. The Read operations successfully retrieved blog information. The Update operation modified the selected blog, and the Filter operation returned matching records. The Delete operation successfully removed the selected blog.

The final test message confirms:

```
CRUD API TESTS COMPLETED
```

12. Advantages

The implemented system provides the following advantages:

- Simple blog management through REST APIs.
- MongoDB-based data storage.
- Easy creation and modification of blog records.
- JSON-based API communication.
- Separate route structure for blog operations.
- Easy testing through the provided test script.
- Automatic database-generated IDs and timestamps.

13. Future Enhancements

The current implementation can be extended with additional features in the future, such as:

- User authentication and authorization.
- Blog image or file upload functionality.
- Advanced search and filtering.
- Pagination for large numbers of blogs.
- Blog comments and likes.
- AI-assisted content generation and summarization.
- API documentation using Swagger/OpenAPI.
- Frontend integration.

These are proposed enhancements and are not part of the functionality demonstrated in the current code and test output.

14. Conclusion

The AI BlogNest API successfully implements a backend system for managing blog posts using Node.js, Express.js, MongoDB, and Mongoose. The application provides the basic operations required for blog management, including creating, retrieving, updating, filtering, and deleting blog records.

The API was tested through a dedicated test script, and the complete CRUD workflow was successfully executed. The results demonstrate that the backend can communicate with MongoDB and perform the required blog data operations.

Overall, the AI BlogNest API provides a structured foundation for a blog management backend and can be further enhanced with additional features such as authentication, advanced search, file handling, and AI-assisted blog functionality.